Secondary Marketplace - Full Stack Assignment

Overview

Build a **Secondary Marketplace** for trading digital securities. This is a Next.js 14 full-stack application with an in-house SQLite database. Your task is to complete the marketplace functionality where users can browse assets, place buy/sell orders, manage their portfolio, and view order history.

Time Expectation

Spend a reasonable amount of time. Focus on quality over quantity. Record a **5-minute screen recording** demonstrating what you achieved.

What's Already Built (DO NOT modify auth flow)

- User authentication (signup, login, email verification)
- Account setup / onboarding
- Banking: deposit funds, payment methods
- Basic portfolio view
- Database schema with trading tables
- Theme and layout components
- A starter `GET /api/trading/assets` API
- An order matching engine (`lib/matchingEngine.ts`)

What You Need to Build

1. Asset Listing Page (`/investing/secondary-trading`)

Display available assets with filtering and search. Navigate to detail page on click.

2. Asset Detail Page (`/investing/secondary-trading/[id]`)

Show asset info, price chart, order book, and an order placement form. Display user's orders and positions.

3. Trading API Routes

Build the backend APIs for placing orders, cancelling orders, and fetching user trading data. The database tables and matching engine are provided — you design the API flow and business logic (validation, balance checks, position updates, etc.).

4. Portfolio Integration

Connect trading to the portfolio — show holdings, order history, cash balance updates.

5. UX Enhancements (Bonus)

Loading states, error handling, responsive design, confirmations, validation — any improvements you think matter.

Key Resources

Data

- **5 trading assets** in `data/secondaryTradingAssets.json` — each with price data, daily OHLCV history, company info
- **Order book template** in the same file at `templates.orderBook` — multiply `priceMultiplier × asset.basePrice` for display prices

What's Provided

Resource	Description
`GET /api/trading/assets`	Starter API — returns all 5 assets (enhance with filtering)
`lib/matchingEngine.ts`	Order matching engine — `matchOrder()` matches buy/sell orders and updates positions
`lib/db.ts`	Database schema with trading tables (orders, trades, holdings, balances)
`lib/auth.ts`	Server-side auth helper: `getAuthUserId(request)`
`lib/api.ts`	Axios client for frontend API calls (auto-injects auth token)
`lib/investmentUtils.ts`	`formatCurrency`, `slugify`, `getSeededColor`, chart helpers
`contexts/AuthContext.tsx`	`useAuth()` hook

Database Tables (schema in `lib/db.ts`)

- `trading_orders` — buy/sell orders (symbol, side, quantity, price, status)
- `trading_trades` — matched trades between orders
- `trading_holdings` — user positions (symbol, shares, avg_cost)
- `trading_balances` — user cash balance (seeded with $1,000 on signup)

Quick Start

```
npm install
npm run dev        # http://localhost:3000
```

1. Sign up at `/auth`, complete account setup
2. Deposit funds at `/account/banking`
3. Build the marketplace at `/investing/secondary-trading`

Next.js Primer (if new to Next.js)

- **Pages**: `page.tsx` files under `app/` — folder path = URL route
- **API routes**: `route.ts` files under `app/api/` — export `GET`, `POST` functions
- **Client components**: Add `'use client'` at top for React hooks / browser APIs
- **Path aliases**: `@/` = project root (e.g., `import api from '@/lib/api'`)

Tech Stack

Next.js 14, React 18, Material-UI 5, TypeScript, SQLite (better-sqlite3), Axios

Guidelines

- Use AI tools (encouraged!)
- Focus on functionality first, polish second
- Test end-to-end
- Look at real trading platforms for inspiration

Submission

1. **Create a public Git repository** — Push this project to your personal GitHub (or GitLab/Bitbucket) account as a **public** repository
2. **Add a `SOLUTION.md`** file at the project root summarizing:
   - What you built and how it works
   - Key technical decisions and trade-offs
   - Anything you'd improve or do differently with more time
3. **Record a ~5-minute screen recording** demonstrating your work — walk through the features, explain your approach, and highlight anything noteworthy. You can upload the video directly to the repository or use a shareable platform (Loom, Google Drive, YouTube unlisted, etc.) and include the link in your `SOLUTION.md`
4. **Share your submission** — Reply to the assignment email with the link to your Git repository

> ***Note:*** *Make sure `node_modules/` and `data/tzero.db` are in `.gitignore` before pushing. Do **not** commit the SQLite database file or dependency folders.*